

第四节：Tomasulo 算法

乱序执行、投机执行、ROB 与重命名

4.0 两类冒险，两套机制

顺序流水线在数据没到位时会让后面无关指令一起停住。Tomasulo 要让执行顺序和取指顺序脱钩。

冒险	处理机制
RAW 数据冒险	操作数产生时间晚于消费时间，用乱序执行处理，让后面无依赖指令先算。
控制冒险	分支结果产生时间晚于取指时间，用投机执行处理，先按预测路径走，错了再撤销。

两套机制共用 ROB：乱序执行靠 ROB 保证写回顺序，投机执行靠 ROB 撤销错误路径。

A. 乱序执行

保留站负责“谁能先算”，ROB 负责“谁能真正生效”。

4.1 保留站

保留站是等待执行的指令暂存区，每个功能单元前面挂若干个 RS。

字段	含义
----	----

Op	要做什么运算
----	--------

Vj/Vk	两个操作数的值，已就绪时填
-------	---------------

Qj/Qk	两个操作数的生产者标签，未就绪时填
-------	-------------------

Dest	结果最终要交给谁
------	----------

判据：两个操作数标签都清空，说明数据齐了，可以执行。

4.1 保留站例子

指令序列

I1: div x5, x6, x7 # 除法, 耗时长

I2: add x8, x5, x9 # 依赖 I1 的 x5

I3: add x5, x10, x11 # 又写 x5, 但操作数早就绪

指令	RS	Vj	Vk	Qj	Qk
I1	RS1	Reg[x6]	Reg[x7]	空	空
I2	RS2	-	Reg[x9]	RS1	空
I3	RS3	Reg[x10]	Reg[x11]	空	空

I3 的两个操作数都就绪, 可以先执行; 它不需要等排在前面的 I1。

4.1 乱序执行周期表

设 div 执行 4 周期，add 执行 1 周期。

指令/周期	1	2	3	4	5	6
I1: div x5,x6,x7	Issue	Execute	Execute	Execute	Write Result	Commit
I2: add x8,x5,x9	-	Issue, 等 RS1	等 RS1	等 RS1	认领广播	Execute
I3: add x5,x10,x11	-	-	Issue	Execute	Write Result	等提交顺序

执行顺序由操作数是否就绪决定，不由程序位置直接决定。

4.2 寄存器状态表

保留站记录某条指令等谁；寄存器状态表记录每个寄存器目前该等谁。

情况	Issue 时的动作
没有指令正在写这个寄存器	把寄存器堆里的值抄进 RS 的 V_j/V_k
已有指令正在写这个寄存器	把生产者标签抄进 RS 的 Q_j/Q_k
本指令要写某寄存器	把自己的标签写进寄存器状态表

4.2 状态表覆盖

接上例，I3 发布时会把 x5 的生产者从 I1 改写成 I3。

时刻	事件	x5 当前生产者
I1 发布	I1 将写 x5	I1 / RS1
I2 发布	I2 读 x5，把等待对象锁定为 I1	I1 / RS1
I3 发布	I3 也将写 x5，覆盖状态表	I3 / RS3

I2 已经把等待对象拷进自己的 RS，后续状态表覆盖不会改变 I2 的依赖。

4.2 CDB

CDB 广播的是：

{生产者标签, 值}

- 所有正在等这个标签的 RS 收到值，清空等待标签。
- 状态表条目如果仍然等这个标签，也清空。
- 如果状态表已经被后面的指令覆盖成别的标签，就不会被这次广播碰到。

匹配靠标签，不靠寄存器号。

4.3 WAW 冒险

继续 div/add/add 例子。若算完就直接写寄存器堆：

- I3 是 add，执行快，先写 x5。
- I1 是 div，执行慢，后写 x5。
- 程序顺序里 I3 在 I1 后，最终 x5 应该是 I3 的结果。

迟到的 I1 覆盖了 I3，这就是 WAW。

4.3 把两个时刻分开

WAW 的根源是把两个时刻绑在一起：

完成计算的时刻 = 结果生效的时刻

解决方式：

- 先记录“我算完了，结果是什么”。
- 不立刻写寄存器堆。
- 等按程序顺序轮到自己提交时，再让结果生效。

4.4 ROB: 乱序侧

ROB 是按发布顺序排队的循环队列。

字段	含义
----	----

目的地	这条指令要写什么寄存器
-----	-------------

value	算出来但尚未生效的结果
-------	-------------

ready	结果是否已经就绪
-------	----------

硬规则：只有队首条目才允许提交；提交这一刻才真正更新寄存器堆。

4.4 ROB 消除 WAW

周期	ROB1(I1 div)	ROB2(I2 add)	ROB3(I3 add)	队首动作
I3 算完	未 ready	等 I1	ready=I3 结果	队首 ROB1 未 ready, ROB3 不能提交
I1 算完	ready=I1 结果	可继续执行	ready=I3 结果	ROB1 提交, x5 写 I1 结果
下一步	已出队	ready=I2 结果	ready=I3 结果	ROB2 提交, x8 写 I2 结果
再下一步	-	已出队	ready=I3 结果	ROB3 提交, x5 写 I3 结果

最终 x5 的生效顺序与程序顺序一致。

B. 投机执行

先按预测路径走；结果只进 ROB，确认路径正确后才生效。

4.5 分支预测

分支预测是在分支比较结果没出来时，先猜方向和目标地址，让取指不要停。

预测不跳转

0x100: beq x5, x6, 16 # 若相等跳到 0x114

0x104: add x7, x8, x9 # 预测路径

0x108: sub x10, x11, x12 # 预测路径

0x114: addi x13, x14, 1 # 真跳转目标

预测不跳转时，取指继续拿 0x104 和 0x108。

4.6 投机执行与撤销

按预测路径继续取指、译码、发布、执行，就是投机执行。

关键约束：这些指令的结果只写进 ROB，不写进寄存器堆或内存。

如果错误路径上的 `add` 直接写了 `x7`，而 `beq` 后来才发现应该跳转，`x7` 就被本不该存在的指令污染了。

4.6 投机执行周期表

设 beq 预测不跳转，但实际应该跳转。

指令/周期	1	2	3	4
beq(ROB1)	Issue	Execute	WR: 发现实际跳转	Commit: 确认分支
add(ROB2)	-	Issue	Execute	结果留在 ROB, 等待命运
sub(ROB3)	-	-	Issue	Execute

预测路径已经执行，但架构状态还没被它们改动。

4.7 ROB: 投机侧

分支预测错误确认后：

- 清空这个分支之后的所有 ROB 条目。
- 清空这些条目占用的保留站和 LSQ 位置。
- PC 重新定向到正确目标。
- 分支本身按正常规则提交。

错误路径从未真正写过寄存器堆或内存，所以撤销不会留下痕迹。

4.7 嵌套投机路径

如果一个分支还没解决，预测路径上又出现第二个分支，ROB 需要区分：

- 哪些条目属于第一个分支之后的预测路径。
- 哪些条目又属于第二个分支之后的预测路径。
- 某个分支预测错时，应该清空哪一段年轻条目。

这是 working Tomasulo 中投机执行真正需要面对的状态管理问题。

ROB 把“结果算出来”和“结果生效”分开。

4.8 ROB 与 LSQ 的分工

结构	分工
----	----

ROB	靠严格顺序提交，同时收拾 WAW 和分支预测错误。
LSQ	判断 load/store 是否有地址冲突，管理 store 延迟写内存。

内存地址是运行时算出来的，两条访存是否真的冲突，要等地址出来才知道。

4.8 LSQ 例子

访存序列

I1: sw x5, 0(x6) # 地址 0x200

I2: lw x7, 0(x8) # 地址也是 0x200

- 如果 I1 地址还没算出来，I2 不能贸然认为安全。
- 如果 I1 地址已知且冲突，I2 要等待，或从 I1 做 store-to-load forwarding。
- store 的真正写内存动作延迟到提交。

4.8 LSQ 周期表

指令/周期	1	2	3	4	5
I1: sw x5,0(x6)	Issue	算地址 0x200	等数据/ready	Commit 写内存	-
I2: lw x7,0(x8)	-	Issue	算地址 0x200	等 I1 或转发	得到 x7

LSQ 的底线：地址不确定时，不能把内存访问当成无冲突。

标签机制本身就是 Tomasulo 的寄存器重命名。

4.9 标签机制就是重命名

重命名把“逻辑寄存器 x5”和“这次写 x5 的具体那条指令”绑在一起。

现象	含义
I2 发布时锁定 I1 的标签	I2 等的是 I1 这次计算，不是 x5 这个名字
I3 后来覆盖状态表	只影响更年轻的读 x5 指令
两次写 x5 有两个标签	WAW 被拆成两个独立版本

4.9 WAR

例子

I1: add x5, x6, x7 # 读旧的 x6、x7

I2: sub x6, x8, x9 # 写 x6

I1 发布时已经把 x6、x7 的值提前抄进自己的操作数字段。

之后 I2 无论多早写完，都动不了 I1 已经拿到的这份拷贝。

WAR 因此在 Tomasulo 里天然不存在。

4.10 一条指令的生命周期

指令

l1: div x5, x6, x7

阶段	主要结构
发布 Issue	状态表、RS、ROB
执行 Execute	RS、功能单元
写回 Write Result	CDB、RS、状态表、ROB
提交 Commit	ROB、寄存器堆或内存

4.10 发布

发布 I1: div x5,x6,x7 时:

- 查寄存器状态表。
- x6、x7 都没人等，直接把寄存器堆里的值抄进 RS。
- 在功能单元前排一个 RS 位置。
- 在 ROB 尾部占一个条目，记录目的寄存器是 x5，结果字段暂空。
- 把状态表里 x5 的生产者改成本指令标签。

4.10 执行

执行阶段：

- RS 判据满足：两个操作数都是值，不是标签。
- 功能单元开始算除法。
- 除法耗时长，但不影响后面无关指令发布或执行。

这就是乱序执行的效果：等待留在局部结构里，不堵住整条指令流。

4.10 写回

写回阶段：

- 除法算完，结果通过 CDB 广播。
- 广播内容是“本指令标签算出了这个值”。
- 所有等这个标签的 RS 和状态表条目收到值。
- ROB 里对应条目填入结果，标记为 ready。

此刻寄存器堆里的 x5 还没变。

4.10 提交

提交阶段：

- 等 I1 到达 ROB 队首，才真正把结果写进寄存器堆。
- 如果是分支，提交动作是确认预测正确或触发撤销。
- 如果是 store，提交动作是真正写内存。

ROB 的顺序规则保证 WAW 不出错、投机路径可撤销、内存和寄存器堆看到程序顺序下的结果。